ISA5810 DM2026 Lab 1 — Master Notebook Questions

Fill in this file as you work through DM2026-Lab1-Master.ipynb. Submit this completed file converted to PDF together with the rest of your Lab 1 work.

Please do not change the style of this document.

Question numbers 1–10 follow the same order as the notebook (top to bottom).

Student Information

Please complete the fields below. For **English Name** and **Chinese Name**, you may fill in **only one** if you do not have the other.

- **English Name: Ian**
- **Chinese Name: 范祥彥**
- **Email: yesyous970194@gapp.nthu.edu.tw**
- **Student ID: 114065525**
- **University: National Tsing Hua University**

Instructions

These questions go with the Master Jupyter notebook (DM2026-Lab1-Master.ipynb). Each question is tied to a specific notebook section. Open the notebook, find **Question N (take home)**, and write your answer here.

We **strongly** suggest running the notebook to learn the lab, but the TAs will **not** audit whether you executed every cell to answer the questions in this document.

How to answer each question

1. Find the **notebook section** listed under the question (use the Table of Contents if needed).
2. Read the task; change parameters when asked (e.g. stop_words, thresholds, k, minSup).
3. Write a clear answer in the **Answer** space below each question.
4. Leave the question text unchanged; write only in the answer and disclosure fields.

Sources, honesty, and AI use

Acceptable vs. Unacceptable AI Usage: Generative AI should serve as an interactive tutor to guide your understanding, not as an automated solver completing the assignment for you.

- **Permitted (AI as Guide & Editor):**
  - Asking AI to explain unfamiliar theoretical concepts, discuss algorithmic trade-offs, or troubleshoot runtime errors in the notebook.
  - Polishing the grammar, phrasing, and readability of your own original, self-written drafts.

- **Prohibited (AI as Solver):**
  - Copying assignment questions directly into an AI tool to generate answers.
  - Using AI to fabricate experimental results or observations without executing the code in DM2026-Lab1-Master.ipynb.
  - Submitting machine-generated prose wholesale.

- **External material:** If you use websites, books, papers, or documentation beyond the notebook, cite them with **clickable links** (or DOI/ISBN when no URL exists).

- **When AI was used:** Your answer is acceptable if you (1) **disclose** AI use and (2) include **at least one external link** that validates the information in your answer (official docs, textbook, paper, or reputable tutorial — not the AI chat alone as the only source).

- **When AI was not used:** You may write "No" under **AI used** or leave it blank only if you truly did not use AI for that question. References are still encouraged if you looked anything up.

Under each question, complete:

- **AI used for this answer:** Yes
- **If yes, how:** (e.g. brainstorming, grammar, concept explanation)
- **External references (links):** *(required when AI = Yes; encouraged otherwise)*

Write in English. Keep answers concise and specific.

Questions

Question 1: Data Filtering & Domain Relevance

Notebook: Section 3.2 — Question 1 (take home)

Task: Filter the dataframe **X** for the category **sci.med** and check its length.

Questions:

1. How many documents are in this specific category?
2. From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

Answer:

1. There are 594 documents in the category of sci.med using the statement of category_name == 'sci.med' to get.

2. Because it is important for us to classify homogeneous groups through their common value in certain columns. If we do not isolate specific categories first, the support of patterns that are frequent within sci.med would be diluted across all 2257 documents and might fall below the minSup threshold. Moreover, if we mine with all documents, the mined patterns would be generic terms across all four categories, which cannot represent the specific feature of the domain.

AI used for this answer: Yes

If yes, how: Claude explained the underlying concepts, helped me interpret the outputs of my own notebook runs, and suggested wording/grammar edits on my draft. I ran all code and verified the numbers myself.

External references (links):

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.loc.html

https://link.springer.com/book/10.1007/978-3-319-07821-2

Question 2: Handling Missing Data

Notebook: Section 4.1 — Question 2 (take home) (after the **dropna()** demonstration)

Task: In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.

Question: If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative so the text data can still be utilized.

Answer: If we simply use dropna(), we will mistakenly diminish the recoverable rows, which is bad for statistical efficacy and model performance. Because they are only missing the label but with valid text, so dropna() will drop the usable data. Moreover, if the label missing does not occur at random, dropping also leads to distribution bias. We should

consider recovering this type of data by training a text classifier with the 60% complete data and use it to predict the missing data of those 40% incomplete data.

AI used for this answer: Yes

If yes, how: Claude explained the underlying concepts, helped me interpret the outputs of my own notebook runs, and suggested wording/grammar edits on my draft. I ran all code and verified the numbers myself.

External references (links):

https://pandas.pydata.org/docs/user_guide/missing_data.html

https://scikit-learn.org/stable/modules/impute.html

Question 3: Data Sampling Risks

Notebook: Section 5.1 — Question 3 (take home)

Task: In Section 5.1, the code uses **X_sample = X.sample(n=1000)** to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.

Questions:

1. Do the exact category proportions perfectly match the original?
2. What is the specific risk of simple random sampling on a highly imbalanced dataset?

Answer: 1. No. The proportions are close but not identical: simple random sampling only guarantees that the expected proportions equal the population's, so the sampled classes deviate by 0.2 to 1.9 percentage points (e.g., comp.graphics 25.9% → 27.8%, soc.religion.christian 26.5% → 24.8%), which even changed the ranking of the classes. 2. Here all four classes exceed 20% and the deviation already reaches ~2 points. If a class made up only 1–2% of the data, the same random fluctuation could cut its share in half or remove it from the sample entirely, so the minority class would be under-represented or missing. Stratified sampling, which samples each class in proportion, avoids this.

AI used for this answer: Yes

If yes, how: Claude explained the underlying concepts, helped me interpret the outputs of my own notebook runs, and suggested wording/grammar edits on my draft. I ran all code and verified the numbers myself.

External references (links):

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.sample.html

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

---

Question 4: Text Vectorization and Matrix Sparsity

Notebook: Section 5.3 — Question 4 (take home)

Task: In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.

Questions:

1. Does the total number of features (columns) increase or decrease?
2. Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?

Answer: 1. The total number of features (columns) increases by 306 columns from 35482 to 35788. 2. The sparsity decreases by about 0.13 percentage points from 99.68% to 99.55%, because stopwords appear in almost every document, the 306 added columns are mostly non-zero, which raises the overall share of non-zero cells. We can say that stopwords make the matrix more dense, but they are not so informative for classifying, so we usually filter them.

AI used for this answer: Yes

If yes, how: Claude explained the underlying concepts, helped me interpret the outputs of my own notebook runs, and suggested wording/grammar edits on my draft. I ran all code and verified the numbers myself.

External references (links):

https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html

---

Question 5: Feature Selection — Variance Thresholding

Notebook: Section 5.3.1 — Question 5 (take home)

Task: In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0, threshold = 0.01, and threshold = 0.05**. Record the number of features marked as **keep == True**.

Questions:

1. What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?
2. In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?
3. Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

Answer: 1. The vocabulary shrinks from 35,482 terms (0.0) to 5,089 (0.01) and only 1,491 (0.05), about 4% of the original. The total size of the vocabulary decline dramatically, this phenomenon suggest that most of the words have low variance, and none of the words dropped means that the variance of every words are greater than zero with and appear at least one time. 2. Words with very low document frequency, i.e., terms that appear in only one or a few documents and are 0 everywhere else. The lowest-variance removed terms all have variance 0.000443, which equals (1/2257)(1 − 1/2257), the variance of a word that appears once in a single document. Typos, names, scattered numbers, no meaning string. 3. Because those kind of words cannot help identifying documents, and not informative for classifying and grouping. Most of those words are typos of rare names which add the noise on the data, so drop them can reduce noises and avoid overfitting. The threshold of the variance is unsupervised, they are not recognized by labels so that data leakage will not happen. However, "safe" only holds for thresholds near zero: 0.05 removed 96% of the vocabulary and may also drop rare but discriminative domain terms, so the threshold should be chosen by inspecting what is removed.

Threshold	**Number of terms kept (keep == True)**
0.0	35482
0.01	5089
0.05	1491

AI used for this answer: Yes

If yes, how: Claude explained the underlying concepts, helped me interpret the outputs of my own notebook runs, and suggested wording/grammar edits on my draft. I ran all code and verified the numbers myself.

External references (links):

https://scikit-learn.org/stable/modules/feature_selection.html#removing-features-with-low-variance

---

Question 6: Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)

Notebook: Section 5.3.2 — Question 6 (take home)

Task: Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.

Questions:

1. Are the top-correlated words generic stop words or domain-specific terminology for graphics?
2. What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?
3. In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

Answer: 1. Neither list is dominated by generic stop words, but they differ: Pearson's top 10 are mostly not stop words but also not graphics terminology: conversational words such as thanks, hi, looking, keywords and the header term host, which reflect that comp.graphics posts are typically help requests; only vga, video and card are domain terms. The top-10 of the Spearman are almost domain terminology like graphics, image, 3d, files, program, windows. Negative correlation words also have information, people, god, writes are prevalent at religious discussion and citing posts, which is negative correlated with comp.graphics and rely on no appearance to be classified. 2. Pearson directly uses raw value to count, it is evaluated as linear relationship, which is sensitive to size and extreme values. Spearman converts the value into ranking and using ranking to calculate Pearson, which evaluates monotonic relationship and insensitive to extreme values. 3. The word graphics appears in most comp.graphics posts but rarely in other classes, graphics ranks 1st by Spearman (0.521) but only 13th by Pearson (0.178). If we observe ranking we can see that the monotonic relationship between appearing and being comp.graphics is consistent, which contributes to the high Spearman. However, its distribution is skewed with the high frequency in posts like long FAQ and software list with the frequency of one. Pearson using original frequency to calculate linear relationship, those extreme counts break the linear relationship with the 0/1 label, which lowers the Pearson coefficient.

AI used for this answer: Yes

If yes, how: Claude explained the underlying concepts, helped me interpret the outputs of my own notebook runs, and suggested wording/grammar edits on my draft. I ran all code and verified the numbers myself.

External references (links):

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.pearsonr.html

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.spearmanr.html

---

Question 7: Frequent Pattern Mining — Algorithm & Parameter Dynamics

Notebook: Section 5.4.2 — Question 7 (take home)

Task: In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100 and k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup=9 and minSup= 3**.

Questions:

1. How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?
2. How does the parameter k in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?
3. Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

Answer (1) — Runtime and pattern count when lowering minSup:

minSup drops from 9 to 3, number of patterns increase from 710 to 9547, and runtime increases from 0.07 seconds to 2.64 seconds, which means runtime grows more quickly than the number of patterns. This is because when threshold drops, more candidate need to expand in FP-tree and observe whether it is maximal, and the combination account is exponentially grows.

Answer (2) — Top-K k vs MaxFPGrowth minSup:

minSup is absolute threshold, we do not know the output number in advance. (minSup = 9 → 710, minSup = 3 → 9,547) k is a ranking threshold: the output size is fixed at k, while the support cutoff is decided by the data (support 21 for k = 100, 16 for k = 500). So k can control the output size more easily, we do not need to know the support distribution. Using minSup need some concept to the data, setting it wrong will lead to null mining or exploded numbers.

Answer (3) — Maximal vs standard frequent patterns:

A maximal frequent pattern is frequent itself, and none of its supersets is frequent. The reason of simplifying is the character of Apriori, the subset of frequent pattern is also frequent. So we only need to memorize the biggest one, once the subsets implicated within

it we do not need to list one by one. For example, the FAE suggest that rank 95 to 99 words of queens sea Bronx, sea queens, sea Bronx, sea sank… are overlapped subsets all with support 21, maximal only keep the longest one like queens sea bronx and omit the behind, but the trade-off is that we will lost exact support of the subset(the closed patterns can keep this information).

AI used for this answer: Yes

If yes, how: Claude explained the underlying concepts, helped me interpret the outputs of my own notebook runs, and suggested wording/grammar edits on my draft. I ran all code and verified the numbers myself. It also helped me troubleshoot runtime errors (missing output folders, UTF-8 encoding on Windows).

External references (links): https://pami-1.readthedocs.io/

Question 8: Dimensionality Reduction — Motivation

Notebook: Section 5.5 — Question 8 (take home) (before PCA/t-SNE code)

Task: After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).

Questions:

1. If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
2. What specific category of data mining algorithms (referenced in the helper files) must be applied first?

Answer: 1. X_counts has 35482 columns and every document is one point in 35482 dimension space, but scatter diagram only draws two to three axes. Moreover, the 99.7% of the matrix is 0, in this high dimensions, the distance of the dots is nearly the same, so we can not see the clustering. 2. We need to do dimensionality reduction first like PCA. This is implemented in helpers/visualization_helpers.py, where compute_dimred_2d_for_filters runs PCA, t-SNE and UMAP.

AI used for this answer: Yes

If yes, how: Claude explained the underlying concepts, helped me interpret the outputs of my own notebook runs, and suggested wording/grammar edits on my draft. I ran all code and verified the numbers myself.

External references (links): https://scikit-learn.org/stable/modules/manifold.html

---

Question 9: Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)

Notebook: Section 5.5 — Question 9 (take home) (after 2D/3D plots)

Task: In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.

Questions:

1. Which method **(PCA or t-SNE/UMAP**) shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
2. Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
3. Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

Answer:

1. UMAP can identify clusters, PCA seriously overlapped in the center, and the 3D t-SNE lies in between. In UMAP of sci.med the black cluster is escaped as an isolated island and atheism extend to the other side. PCA almost every dot squeezed to one pillar with only a few dots drift far away. t-SNE is oval with only a little block.
2. PCA is linear and it keep the direction of the whole variance, so it is dominated by a few extreme documents, and other documents is squeezed to one pillar. t-SNE/UMAP is non-linear that keeps only neighborhood relationship, so we can cluster same class documents despite there is not a straight line between borders.
3. t-SNE only preserves local neighborhoods: points that are close in the high-dimensional space stay close in the plo*t*, the distance between clusters and cluster size has no meaning.

AI used for this answer: Yes

If yes, how: Claude explained the underlying concepts, helped me interpret the outputs of my own notebook runs, and suggested wording/grammar edits on my draft. I ran all code and verified the numbers myself. It also helped me troubleshoot runtime errors (missing output folders, UTF-8 encoding on Windows).

External references (links):

https://distill.pub/2016/misread-tsne/

https://umap-learn.readthedocs.io/

Question 10: One-Hot Binarization

Notebook: Section 5.6 — Question 10 (take home)

Task: In Section 3.1, examine **X['category']** (integers 0, 1, 2, 3). Then look at the output of **LabelBinarizer** in Section 5.6, where each category is converted into a binary vector (e.g., [1, 0, 0, 0]).

Questions:

1. How many binary columns are created to represent the 4 categories?
2. If an algorithm (like linear regression or KNN) is given the raw integer column **(0, 1, 2, 3)** as input, what false mathematical assumption does it make about the categories?
3. How does one-hot binarization eliminate this false assumption?

Answer: 1. Every list have four elements, so there are four binary columns, one per class. The document 0 with the number 1(comp.graphics) and with list of [0, 1, 0, 0]. The document 2 with the number 3(soc.religion.christian) and with list of [0, 0, 0, 1]. 2. If we encode with integer, the algorithm will think comp.graphics(1) and sci.med(2) is close, but alt.atheism(0) and christian(3) is farthest. Moreover, the algorithm will think 3 is greater than 1, or mistakenly think 2 is just located between the central of 1 and 3. In conclusion, these kind of pairs do not have this ordinal or distancing relationship. KNN computes the distance, linear regression computes coefficient, these two will mistakenly make this fake relationship true. 3. One-hot makes every class occupy one axis, and any two vector distance is √2, so there is no ordinal relationship.

AI used for this answer: Yes

If yes, how: Claude explained the underlying concepts, helped me interpret the outputs of my own notebook runs, and suggested wording/grammar edits on my draft. I ran all code and verified the numbers myself.

External references (links):

https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelBinarizer.html

---